
PROBABILISTIC FORECASTING VIA AUTOREGRESSIVE FLOW MATCHING

Anonymous authors
Anonymous submission

ABSTRACT

We introduce FLOWTIME, a generative model for probabilistic forecasting of multivariate timeseries. We cast forecasting as sampling from a learned conditional distribution over future trajectories given a finite history of observations and optional covariates, and we factorize this distribution autoregressively across the forecast horizon into a sequence of per-step conditionals, each realized by a single shared flow that transports a standard base distribution onto the predictive distribution of the next observation conditioned on the recent context. Instantiating these transports within the flow matching (FM) framework affords scalable, simulation-free estimation of the underlying velocity fields. The decisive consequence of this design is that transport is confined to a single time step rather than an entire horizon-long window: the dimensionality of each learning subproblem is reduced, and the model’s inductive bias is aligned with the sequential structure of the forecasting task. Coupling the autoregressive factorization with the FM objective thereby combines two families of desirable properties: the reliable extrapolation beyond the training horizon, compact parameterization, and calibrated uncertainty characteristic of autoregressive models, together with the expressive, multi-modal per-step predictive distributions afforded by modern transport-based generative models. On stochastic dynamical systems, a controlled ablation isolates the autoregressive factorization and shows that it improves point accuracy over an otherwise identical non-autoregressive flow-matching model and, tellingly, preserves that advantage when forecasting beyond the training horizon; on five real-world benchmarks, FLOWTIME attains state-of-the-art or highly competitive CRPS relative to strong statistical, deep-learning, diffusion, and flow-matching baselines. Together these results establish autoregressive flow matching as an effective and practical approach to probabilistic timeseries forecasting.

1 INTRODUCTION

Probabilistic timeseries forecasting—inferring, from past observations, how the dynamics of a system will evolve together with a calibrated account of the associated uncertainty—is a foundational problem in modern machine learning and a prerequisite for principled decision-making across an exceptionally broad range of scientific, industrial, and societal domains (Lim & Zohren, 2021; Dama & Sinoquet, 2021; Ye et al., 2024). In such settings the operative requirement is not a single accurate point prediction but a faithful predictive distribution over future trajectories, from which risk-sensitive quantities and calibrated uncertainty intervals can be derived.

A prominent and increasingly influential line of work casts this problem in the language of deep generative modeling (Karl et al., 2016; Rasul et al., 2020; Desai et al., 2021), treating forecasting as the task of sampling from a future probability density conditioned on the past. Within this paradigm, diffusion and score-based generative models (Sohl-Dickstein et al., 2015; Ho et al., 2020; Song et al., 2020) have advanced the state of the art across numerous forecasting benchmarks (Rasul et al., 2021; Tashiro et al., 2021; Kolloviev et al., 2023; Meijer & Chen, 2024). This empirical success is nonetheless purchased at a considerable computational price: diffusion-based forecasters are typically costly to train and, because sampling proceeds through long iterative denoising chains, costly to query at inference time.

Flow Matching (FM) (Liu et al., 2022; Albergo et al., 2023; Lipman et al., 2022) has emerged as a compelling generative-modeling paradigm that subsumes diffusion models as a special case while affording markedly greater design freedom. In place of the iterative stochastic denoising steps that diffusion models perform over a discretized trajectory, FM regresses a deterministic velocity field whose induced probability path transports an arbitrary base distribution directly onto the target through a continuous normalizing flow. Because the probability path is prescribed rather than learned—most naturally as a straight, optimal-transport interpolant—FM dispenses with the hand-crafted noise schedules and protracted ancestral sampling chains intrinsic to diffusion, and in doing so attains substantially more efficient training and sampling.

Recent work has brought FM to bear on timeseries modeling and probabilistic forecasting (Tamir et al., 2024; Hu et al., 2024; Kollovich et al., 2024), reporting strong empirical results and improved computational efficiency relative to diffusion-based counterparts. These approaches share a common design decision that, we argue, constrains them: they learn, in a single shot, the conditional distribution of an entire fixed-horizon future window given a fixed-horizon context window. This window-to-window formulation is fast to train and to sample from, but it couples the temporal extent of the forecast to the transport problem the model must solve, and this coupling entails three concrete liabilities. It generalizes poorly to horizons beyond those seen during training; it tends to yield miscalibrated uncertainty estimates; and it burdens the model with an intrinsically high-dimensional optimization problem in which intricate, time-dependent probability paths must be learned jointly along the flow dimension. The last of these difficulties is made explicit by Kollovich et al. (2024), who mitigate it with Gaussian-process-informed priors that simplify and accelerate optimization; that remedy, however, confines their method to univariate forecasting.

We propose FLOWTIME, a conceptually simple yet effective alternative that harnesses FM to learn probabilistic forecasters scaling to high-dimensional multivariate data without recourse to informed priors. Where existing FM-based methods model the entire future window jointly, FLOWTIME factorizes the forecasting problem autoregressively into a sequence of per-step conditional distributions, modeling each future observation conditioned on recent observations and covariates through a single shared flow that transports a standard base distribution onto the predictive distribution of the next observation. This factorization decouples the temporal structure of the forecast from the transport problem solved at each step, and thereby addresses the liabilities of the window-to-window formulation directly: it accommodates arbitrary and variable forecast horizons by construction, extends reliably beyond the training horizon, yields well-calibrated uncertainty estimates, and reduces each learning subproblem to a lower-dimensional, stationary transport that is easier to optimize.

Our contributions are threefold. *(i) Method.* We introduce autoregressive flow matching, which couples an autoregressive, Markovian decomposition across the forecast horizon with a single shared flow-matching transport applied per step, and we specify the corresponding training and sampling procedures (Algorithms 1 and 2). Unlike autoregressive normalizing-flow forecasters, which require the per-step transport to be an invertible map with tractable Jacobian, the shared flow here is learned simulation-free by velocity regression; unlike diffusion- and window-to-window flow-matching forecasters, the transport acts on a single observation rather than an entire horizon-long window, so its dimensionality is decoupled from the forecast length. *(ii) Controlled ablation.* On five stochastic dynamical systems we compare FLOWTIME against a non-autoregressive flow-matching model that is identical apart from the factorization, isolating the factorization as the sole point of variation and showing that it improves point-forecast accuracy on every system and, critically, retains that advantage when forecasting beyond the training horizon. *(iii) Real-world evaluation.* On five real-world multivariate benchmarks from GluonTS, FLOWTIME attains state-of-the-art or highly competitive CRPS relative to strong statistical, deep-learning, diffusion, and flow-matching baselines. Together, these results establish autoregressive flow matching as an effective and practical approach to probabilistic timeseries forecasting.

2 BACKGROUND: FLOW MATCHING

Flow matching (FM) (Lipman et al., 2022) is a generative-modeling paradigm that enables continuous normalizing flows (CNFs) (Chen et al., 2018; Grathwohl et al., 2018) to be trained at a scale previously out of reach. A CNF represents a data transformation as the solution of an ordinary differential equation (ODE), yielding an invertible map with a tractable likelihood; the difficulty

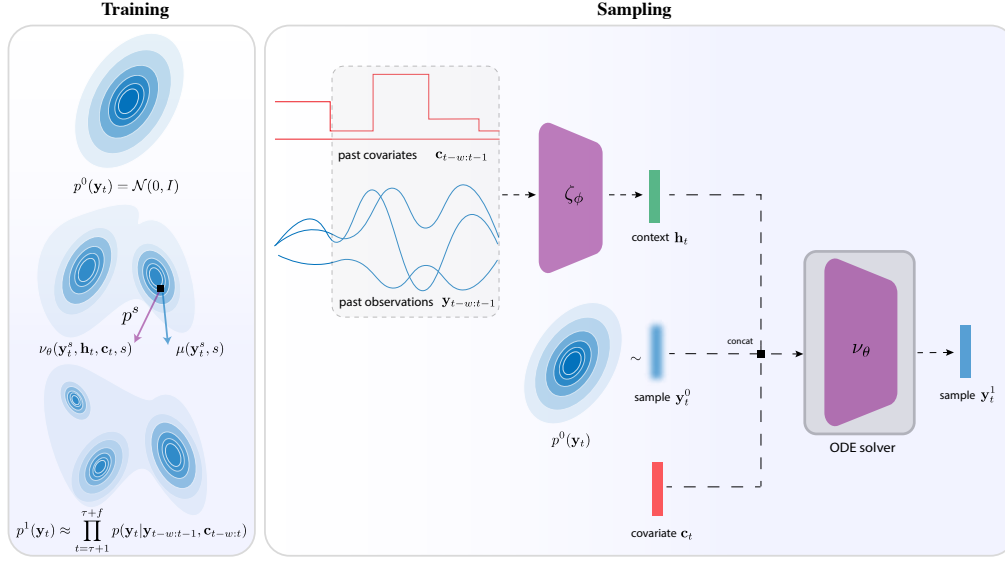


Figure 1: Overview of the training and sampling procedures underlying FLOWTIME. During training, a probability path $(p^s)_{0 \leq s \leq 1}$ interpolating between a base distribution p^0 and the target distribution p^1 is generated by a velocity field μ , and the network ν_θ is trained to regress μ from three inputs: a sample y_t^s drawn from the path at flow step s , a context vector $\mathbf{h}_t$ encoding recent observations and covariates, and the current covariate $\mathbf{c}_t$. At inference, a sample from the target predictive distribution is obtained by drawing from the base distribution p^0 and integrating the learned velocity field with an ODE solver to $s = 1$. The context $\mathbf{h}_t$ is advanced autoregressively across the forecast horizon, so that each future observation is generated conditioned on the previously generated ones.

is that likelihood-based training of a CNF requires simulating the ODE and differentiating through the solver, an operation whose cost grows prohibitively in high dimensions. FM circumvents this bottleneck. We summarize the framework below to fix the notation used throughout.

In common with CNFs, FM seeks a time-dependent diffeomorphic map on the data space Ω , termed a flow $\psi: [0, 1] \times \Omega \rightarrow \Omega$. This flow transports a sample $\mathcal{X}^0 \sim p$ from a source distribution p to a target sample $\mathcal{X}^1 := \psi(\mathcal{X}^0, 1)$ distributed according to a prescribed target q , so that $\mathcal{X}^1 \sim q$. It is defined as the solution of the initial value problem

$$d\psi(\mathcal{X}, s) = \mu(\psi(\mathcal{X}, s), s)ds, \quad \psi(\mathcal{X}, 0) = \mathcal{X}, \quad s \in [0, 1] \quad (1)$$

where the vector field $\mu: [0, 1] \times \Omega \rightarrow \Omega$ specifies the instantaneous velocity of the flow and generates a probability path $(p^s)_{0 \leq s \leq 1}$, a family of distributions over Ω with $p^0 = p$. The generative task therefore reduces to learning a vector field μ whose induced path terminates at the prescribed target, $p^1 = q$.

Whereas a CNF learns this vector field by likelihood maximization—which requires solving, and differentiating through, Eq. (1) at every training step—FM sidesteps simulation entirely by regressing a parametric field ν_θ (with parameters θ) against the analytically known velocity fields of a family of *conditional* probability paths. This substitution yields the tractable conditional flow matching (CFM) objective

$$\mathcal{L}_{\text{CFM}}(\theta) = \mathbb{E}_{s \sim \mathcal{U}[0, 1], \mathcal{X}^1 \sim q(\mathcal{X}^1), \mathcal{X} \sim p^s(\mathcal{X} | \mathcal{X}^1)} \|\nu_\theta(\mathcal{X}, s) - \mu(\mathcal{X}, s | \mathcal{X}^1)\|^2. \quad (2)$$

Once this objective is minimized, samples from q are generated by integrating Eq. (1) forward with the trained field ν_θ . By removing backpropagation through the ODE solver, FM affords the stable, scalable training that has proven decisive in high-dimensional generative tasks such as image and video synthesis (Esser et al., 2024; Polyak et al., 2024). A range of probability-path designs and conditioning strategies has since been developed, unifying a broad class of transport-based generative models within this single framework (Liu et al., 2022; Albergo et al., 2023; Tong et al., 2023).

3 AUTOREGRESSIVE FLOW MATCHING

Problem Setting Let $\mathbf{y}_\tau \in \mathbb{R}^n$ denote an n -dimensional observation at time τ . Given the l most recent measurements up to and including the current one, $\mathbf{Y}_l = \{\mathbf{y}_{\tau-l}, \dots, \mathbf{y}_\tau\}$, the task is to produce probabilistic forecasts of the subsequent f values $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$, optionally conditioned on observed covariates $\mathbf{C} = \{\mathbf{c}_{\tau-l}, \dots, \mathbf{c}_{\tau+f}\}$ with $\mathbf{c}_\tau \in \mathbb{R}^c$. Formally, we learn to sample from the conditional distribution $p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C})$ given a dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$ of m instances. Because the observation $\mathbf{y}_\tau$ is n -dimensional and the covariate term $\mathbf{C}$ is retained throughout, the formulation applies directly to high-dimensional multivariate series and is not subject to the univariate restriction imposed by prior FM-based forecasters.

We factorize this conditional distribution autoregressively across future time steps, invoking a Markov assumption of order w , so that

$$p(\mathbf{Y}_f | \mathbf{Y}_l, \mathbf{C}) = \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \quad (3)$$

where $w \leq l$ is the history size. Because the factors in Eq. (3) share parameters, learning reduces to estimating a single per-step conditional. Departing from the maximum-likelihood estimation conventional for autoregressive models, we estimate this conditional within the flow matching framework (Lipman et al., 2022), so that each factor is realized by a shared transport rather than by an explicit, normalized density.

This autoregressive factorization confers four concrete advantages that jointly motivate our design. First, it substitutes the estimation of a complex, high-dimensional joint distribution over $\mathbb{R}^{f \times n}$ with the substantially simpler subproblem of modeling the per-step conditional $p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ over $\mathbb{R}^n$. Second, because ground-truth histories are available during training, all per-step conditionals can be optimized concurrently under teacher forcing, so that the training computation parallelizes across time steps. Third, the Markov order w exposes an explicit, tunable trade-off between the amount of history the model conditions on and its computational cost. Fourth, the chain of conditional distributions renders the propagation of predictive uncertainty through time explicit, and thereby interpretable.

3.1 CONSTRUCTING A TARGET PROBABILITY PATH

For every future time step $t \in [\tau + 1, \tau + f]$, we construct a target probability path $(p^s(\mathbf{y}_t))_{0 \leq s \leq 1}$ whose endpoints are the standard Gaussian base $p^0(\mathbf{y}_t) = \mathcal{N}(0, I)$ and the data-generating target $p^1(\mathbf{y}_t) \approx \prod_{t=\tau+1}^{\tau+f} p(\mathbf{y}_t | \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \approx p_{\mathcal{D}}$, with $p_{\mathcal{D}}$ the empirical distribution. The defining feature of this construction is that the transport is specified *per time step* rather than over the full horizon: the same base distribution over $\mathbb{R}^n$ is mapped onto each conditional in turn, and the flow dimensionality remains that of a single observation regardless of the forecast length f . The path is generated by a flow $\psi: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ evolving along s according to the ordinary differential equation (ODE)

$$d\psi(\mathbf{y}_t, s) = \mu(\psi(\mathbf{y}_t, s), s)ds, \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t \quad s \in [0, 1] \quad (4)$$

where the vector field $\mu: [0, 1] \times \mathbb{R}^n \rightarrow \mathbb{R}^n$ generating the flow is chosen so that the induced path satisfies the continuity equation.

To render the objective tractable, we express the marginal probability path as a mixture of conditional probability paths,

$$p^s(\mathbf{y}_t) = \int p^s(\mathbf{y}_t | \mathbf{z}) p(\mathbf{z}) d\mathbf{z} \quad (5)$$

where the conditioning variable $\mathbf{z} \sim \pi_{0,1}(\mathbf{z})$ is drawn from an arbitrary coupling $\pi_{0,1}$. Theorem 2 of Lipman et al. (2022) guarantees that the conditional velocity fields $\mu(\mathbf{y}_t, s | \mathbf{z})$ associated with the conditional paths $p^s(\mathbf{y}_t | \mathbf{z})$ integrate, in expectation over $\mathbf{z}$, to the marginal velocity field $\mu(\mathbf{y}_t, s)$. Regressing against the analytically available conditional fields therefore recovers the otherwise intractable marginal field.

Following Tong et al. (2023), we adopt the independent coupling $\mathbf{z} = \{\mathbf{y}_t^0, \mathbf{y}_t^1\}$ with $p(\mathbf{z} = (\mathbf{y}_t^0, \mathbf{y}_t^1)) = p^0 \times p^1$, where $\mathbf{y}_t^0$ and $\mathbf{y}_t^1$ are drawn from p^0 and p^1 , respectively, and define the

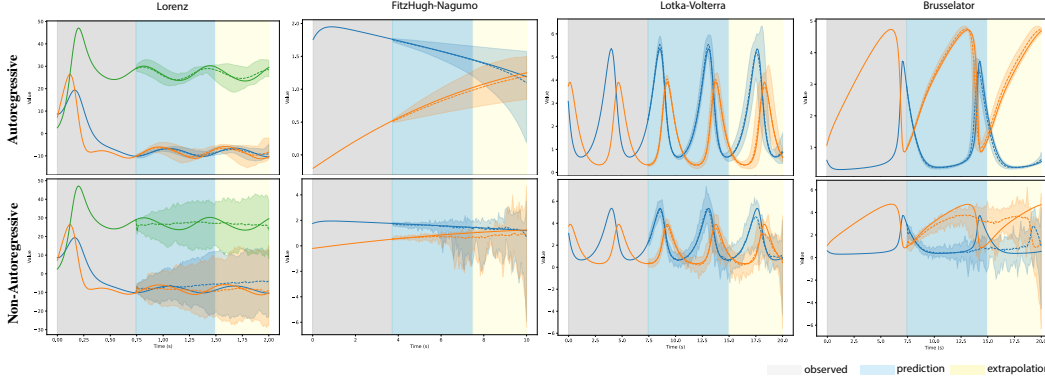


Figure 2: Qualitative forecasts from autoregressive flow matching (FLOWTIME) versus standard, non-autoregressive flow matching on four dynamical systems. Solid lines denote the ground-truth trajectory; dashed lines denote the predictive mean, shaded by the corresponding 95% confidence interval estimated from sampled trajectories. Each panel spans both the prediction regime (horizons seen during training) and the extrapolation regime (horizons withheld entirely during training). The comparison makes visible that the autoregressive factorization continues to track the true dynamics and to maintain a plausibly calibrated predictive interval beyond the training horizon, precisely where the non-autoregressive model departs from the trajectory.

conditional probability path as the Gaussian bridge

$$p^s(\mathbf{y}_t|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I) \quad (6)$$

whose marginals recover the prescribed endpoints, $p^0(\mathbf{y}_t|\mathbf{z}) = p^0(\mathbf{y}_t)$ and $p^1(\mathbf{y}_t|\mathbf{z}) = p^1(\mathbf{y}_t)$, in the limit $\sigma^2 \rightarrow 0$. The associated conditional velocity field is then the constant $\mu(\mathbf{y}_t, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$. This choice yields straight-line probability paths between source and target in Euclidean space together with closed-form conditional velocities—the two properties that make both training and sampling efficient. Once the velocity field is learned, samples from the target conditional $p(\mathbf{y}_t|\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t})$ are obtained by integrating the ODE in Eq. (4) forward to $s = 1$.

3.2 TRAINING

As in the standard FM setting, we learn the velocity field by regressing a neural network ν_θ with parameters θ against the conditional target using the CFM objective of Eq. 2. To make the per-step conditioning tractable, we summarize the context window at time t in a fixed-dimensional context vector $\mathbf{h}_t \in \mathbb{R}^h$, produced by a learned sequence encoder,

$$\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1}) \quad (7)$$

where $\zeta: \mathbb{R}^{w \times n} \times \mathbb{R}^{w \times c} \rightarrow \mathbb{R}^h$ is a sequence encoder with parameters ϕ that compresses the w -step history into $\mathbf{h}_t$. Conditioning the velocity network jointly on $\mathbf{h}_t$ and the current covariate $\mathbf{c}_t$, the learning objective becomes

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{y}_t^s \sim p^s(\mathbf{y}_t|\mathbf{z})} \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2 \quad (8)$$

which we optimize by drawing an observation at a time point t together with its associated context window and covariates from $\mathcal{D}$ and jointly minimizing $\mathcal{L}(\theta, \phi)$ over ϕ and θ by stochastic gradient descent. Because the conditioning history is drawn from ground truth rather than from the model’s own predictions, the scheme is an instance of teacher forcing (Williams & Zipser, 1989): it decouples the per-step subproblems, permits their parallel optimization across all time steps, and precludes error accumulation during training. This training–sampling asymmetry—ground-truth histories at training time versus model-generated histories at inference—is the well-known exposure-bias trade-off of autoregressive models, whose consequences for long-horizon rollouts we examine when discussing the method’s limitations. Algorithm 1 summarizes the resulting training procedure for autoregressive flow matching.

Algorithm 1 Training: Autoregressive Flow Matching.

Require: Dataset $\mathcal{D} = \{(\mathbf{Y}_f^i, \mathbf{Y}_l^i, \mathbf{C}^i)\}_{i=1}^m$, window size w , networks ν_θ, ζ_ϕ

- 1: **while** not converged **do**
- 2: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$ and $(\mathbf{y}_t^1, \mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t}) \sim \mathcal{D}$
- 3: Set $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$
- 4: Sample $s \sim \mathcal{U}(0, 1)$ and $\mathbf{y}_t^s \sim \mathcal{N}((1-s)\mathbf{y}_t^0 + s\mathbf{y}_t^1, \sigma^2 I)$
- 5: Compute target: $\mu(\mathbf{y}_t^s, s|\mathbf{z}) = \mathbf{y}_t^1 - \mathbf{y}_t^0$
- 6: Compute loss: $\mathcal{L}_t = \|\mu(\mathbf{y}_t^s, s|\mathbf{z}) - \nu_\theta(\mathbf{y}_t^s, \mathbf{h}_t, \mathbf{c}_t, s)\|^2$
- 7: Update θ, ϕ
- 8: **end while**
- 9: **return** ν_θ, ζ_ϕ

Algorithm 2 Sampling: Autoregressive Flow Matching.

Require: Past observations $\mathbf{Y}_l$, covariates $\mathbf{C}$, forecast horizon f , window size w , networks ν_θ, ζ_ϕ

- 1: Initialize $\mathbf{Y}_f \leftarrow \mathbf{Y}_l$
- 2: **for** $t = \tau + 1$ **to** $\tau + f$ **do**
- 3: Set context: $\mathbf{h}_t = \zeta_\phi(\mathbf{y}_{t-w:t-1}, \mathbf{c}_{t-w:t-1})$, using the last w entries in $\mathbf{Y}_f$
- 4: Sample $\mathbf{y}_t^0 \sim \mathcal{N}(0, I)$
- 5: Solve $\frac{d\psi(\mathbf{y}_t, s)}{ds} = \nu_\theta(\psi(\mathbf{y}_t, s), \mathbf{h}_t, \mathbf{c}_t, s), \quad \psi(\mathbf{y}_t, 0) = \mathbf{y}_t^0, \quad s \in [0, 1]$
- 6: Append $\psi(\mathbf{y}_t, 1)$ to $\mathbf{Y}_f$
- 7: **end for**
- 8: **return** $\mathbf{Y}_f$

3.3 INFERENCE

At inference time, given new past observations $\mathbf{Y}_l$ and their associated covariates $\mathbf{C}$, we generate probabilistic forecasts over the next f time steps following the sampling procedure of Algorithm 2. Predictions are produced autoregressively under a rolling-window scheme: at each step the context vector is recomputed from the most recent w values—which now include the model’s own previously generated predictions rather than ground-truth observations—and a single observation is sampled by integrating the learned ODE from a fresh draw of the base distribution to $s = 1$. Repeating this procedure over many draws yields an ensemble of trajectories, from which empirical quantiles, and hence calibrated uncertainty estimates, are obtained. Because the forecast horizon f enters only as the number of autoregressive steps, the same trained model produces forecasts of arbitrary length without retraining—a property we exploit directly in the extrapolation experiments of Section 4.1.

4 EXPERIMENTS

Our empirical evaluation is organized around two questions. First, does the autoregressive factorization at the heart of FLOWTIME improve forecasting relative to an otherwise identical non-autoregressive flow-matching model? Second, is the resulting method competitive with established forecasters on real-world data? We address the first question through a controlled ablation on five stochastic dynamical systems (Section 4.1), in which the factorization is the sole point of variation between the two models, and the second through benchmarks against strong statistical, deep-learning, diffusion, and flow-matching baselines on five real-world multivariate datasets (Section 4.1 onward). Throughout, we assess point accuracy and probabilistic calibration jointly, and—on the synthetic systems, where held-out horizons are available—we report performance separately in the prediction and extrapolation regimes so that in-distribution and out-of-distribution behavior can be read off directly.

4.1 DYNAMICAL SYSTEMS

Data We evaluate FLOWTIME on five stochastic dynamical systems that together span oscillatory, chaotic, and limit-cycle regimes: stochastic variants of the Lorenz system (Lorenz, 1963), the FitzHugh–Nagumo model (FitzHugh, 1961), the Lotka–Volterra system Lotka (1925), the Brusselator (Lefever & Prigogine, 1968), and the Van der Pol oscillator (van der Pol, 1926). For each

system, trajectories were generated by numerically integrating the corresponding stochastic differential equations under randomly sampled initial conditions and independent Brownian-motion realizations, so that the learning target is a genuinely stochastic distribution over trajectories rather than a single deterministic path—a setting that directly exercises the model’s capacity to represent multi-modal predictive distributions. Full specifications of the systems and of the data-generation protocol are given in Appendix A.1.

Implementation details We instantiate the context encoder ζ_ϕ as a two-layer bidirectional LSTM with 64 hidden units and the velocity network ν_θ as a three-hidden-layer MLP of width 64. The flow step s is encoded through Fourier positional embeddings with a fixed set of frequencies into a 16-dimensional vector, and the Markov window w is set equal to the prediction length. Optimization uses Adam with a learning rate of 0.003 and a batch size of 128. Training to convergence required approximately 7 minutes on a single NVIDIA A100-SXM4-40GB GPU, a budget that reflects the compactness of the shared per-step transport relative to models that must learn a horizon-wide joint distribution.

Baselines To isolate the contribution of the autoregressive factorization, we compare FLOWTIME against a non-autoregressive flow-matching baseline that models the joint distribution of the entire future window directly, $p^1 = p(\mathbf{Y}_f | \mathbf{Y}_I, \mathbf{C})$, while sharing the flow-matching machinery, conditioning inputs, and training protocol. Because the two models differ *only* in whether the future is factorized autoregressively, the comparison constitutes a clean ablation of that single design choice. The non-autoregressive formulation moreover closely mirrors the window-to-window construction of Kollovich et al. (2024), so the comparison additionally situates FLOWTIME against the dominant prior FM-based strategy for forecasting. Full details of the baseline are provided in Appendix B.

Evaluation To probe extrapolation explicitly, we partition each trajectory into three consecutive segments: an observation window supplied to the model as history, a prediction window used as the training target, and an extrapolation window withheld entirely during training, so that performance on the latter measures generalization to horizons never encountered in training. We quantify probabilistic calibration with the continuous ranked probability score (CRPS) (Winkler et al., 1996) and point accuracy with the normalized root mean square error (NRMSE) (Hyndman & Koehler, 2006), reporting each metric separately for the prediction and extrapolation windows so that in-distribution and out-of-distribution behavior are directly legible. For both metrics, lower values indicate better forecasts. Precise definitions of the metrics are given in Appendix C.

Results Table 1 shows that the autoregressive factorization delivers a consistent improvement in point-forecast accuracy across all five stochastic dynamical systems. In the prediction regime, FLOWTIME attains lower NRMSE than the non-autoregressive baseline on every system without exception; the gains are especially pronounced on the Lorenz (0.017 vs. 0.242) and Brusselator (0.031 vs. 0.658) systems, corresponding to error reductions of 93% and 95%, respectively. On the probabilistic side, FLOWTIME achieves lower CRPS on four of the five systems; the sole exception is the Van der Pol oscillator, on which the non-autoregressive baseline attains better CRPS (0.143 vs. 0.225) even as FLOWTIME retains a substantial NRMSE advantage (0.051 vs. 0.161). Most consequentially, the accuracy advantage does not merely survive the transition to the extrapolation regime but persists uniformly: FLOWTIME improves extrapolation NRMSE over the baseline on all five systems, evidence of a markedly stronger ability to generalize beyond the training horizon. The effect is most striking on the Brusselator, where FLOWTIME reduces extrapolation NRMSE by 98% (0.023 vs. 1.046), and on the Lorenz system, where the non-autoregressive baseline attains marginally better extrapolation CRPS (0.120 vs. 0.137) yet trails by an order of magnitude in extrapolation NRMSE (0.301 vs. 0.048). Taken together, and with the factorization held as the only difference between the two models, these results establish that explicitly modeling temporal dependencies through the autoregressive factorization enables FLOWTIME to track the underlying stochastic dynamics more faithfully, and that this advantage is largest precisely in the demanding out-of-distribution extrapolation setting, where the non-autoregressive alternative deteriorates sharply.

Table 1: Controlled ablation of the autoregressive factorization: autoregressive (FLOWTIME) versus non-autoregressive (Non-AR) flow matching on five stochastic dynamical systems, the two models being identical apart from the factorization. We report test-set CRPS and NRMSE (lower is better) in both the prediction and extrapolation regimes, as mean $\pm$ standard deviation over five random seeds; within each system the better value in a column is shown in **bold**. The autoregressive factorization improves NRMSE over the non-autoregressive baseline on every system in both regimes, and improves CRPS on four of the five systems in the prediction regime; the sole CRPS exception is the Van der Pol oscillator, on which FLOWTIME nonetheless retains a large NRMSE advantage.

System	Factorization	Prediction		Extrapolation	
		CRPS $\downarrow$	NRMSE $\downarrow$	CRPS $\downarrow$	NRMSE $\downarrow$
Lorenz	Non-AR	0.147 $\pm$ 0.012	0.242 $\pm$ 0.018	0.120 $\pm$ 0.009	0.301 $\pm$ 0.024
	FLOWTIME	0.090 $\pm$ 0.008	0.017 $\pm$ 0.003	0.137 $\pm$ 0.011	0.048 $\pm$ 0.006
FitzHugh-Nagumo	Non-AR	0.165 $\pm$ 0.014	0.131 $\pm$ 0.011	0.214 $\pm$ 0.016	0.306 $\pm$ 0.022
	FLOWTIME	0.144 $\pm$ 0.013	0.090 $\pm$ 0.007	0.146 $\pm$ 0.012	0.278 $\pm$ 0.019
Lotka-Volterra	Non-AR	0.130 $\pm$ 0.011	0.266 $\pm$ 0.019	0.136 $\pm$ 0.012	0.502 $\pm$ 0.033
	FLOWTIME	0.126 $\pm$ 0.010	0.189 $\pm$ 0.015	0.123 $\pm$ 0.009	0.339 $\pm$ 0.026
Brusselator	Non-AR	0.217 $\pm$ 0.016	0.658 $\pm$ 0.042	0.238 $\pm$ 0.019	1.046 $\pm$ 0.057
	FLOWTIME	0.125 $\pm$ 0.011	0.031 $\pm$ 0.004	0.109 $\pm$ 0.008	0.023 $\pm$ 0.003
Van der Pol	Non-AR	0.143 $\pm$ 0.012	0.161 $\pm$ 0.014	0.211 $\pm$ 0.018	0.510 $\pm$ 0.031
	FLOWTIME	0.225 $\pm$ 0.017	0.051 $\pm$ 0.005	0.282 $\pm$ 0.021	0.101 $\pm$ 0.009

4.2 REAL-WORLD DATASETS

Data We evaluate FLOWTIME on five widely used real-world forecasting benchmarks from GluonTS (Alexandrov et al., 2020) that vary in dimensionality, sampling frequency (hourly and daily), and domain: Electricity (Lai et al., 2018), Solar (Lai et al., 2018), Exchange (Dua et al., 2017), Traffic (Dua et al., 2017), and Wikipedia (Gasthaus et al., 2019). Spanning 8 to 2000 series and two orders of magnitude in dimensionality, these benchmarks exercise the model across a broad spectrum of multivariate forecasting conditions. Further details are provided in Appendix A.2.

Implementation details For the real-world benchmarks we instantiate the context encoder ζ_ϕ as a three-layer bidirectional LSTM with 64 hidden units and the velocity network ν_θ as a residual network of five residual blocks built from 1-D convolutional layers with gated activation functions (Van Den Oord et al., 2016; Kong et al., 2020). This architecture follows that of Rasul et al. (2021) but employs fewer residual blocks, yielding a deliberately compact model. The flow step s is encoded via Fourier positional embeddings with a fixed set of frequencies into a 32-dimensional vector, and the same embedding scheme encodes the time-dependent covariates $\mathbf{c}$. Optimization uses Adam with a learning rate of 0.001 and a batch size of 128. Depending on the dataset, training to convergence took between 40 and 100 minutes on a single NVIDIA A100-SXM4-40GB GPU, keeping inference and training within a practical budget across the full range of dimensionalities considered.

Baselines We benchmark FLOWTIME against a representative suite of forecasters spanning four methodological families: classical statistical models—Seasonal Naive (SN), AutoARIMA, and AutoETS (Hyndman et al., 2008); deep autoregressive and convolutional models—DeepAR (Salinas et al., 2020) and WaveNet (Van Den Oord et al., 2016); diffusion-based generative models—TSDiff (Kollovich et al., 2023), SSSD (Alcaraz & Strodthoff, 2022), and TimeGrad (Rasul et al., 2021); and the flow-matching forecaster TSFlow (Kollovich et al., 2024), the closest prior method to ours. This breadth measures FLOWTIME against the strongest contemporary approaches, including state-of-the-art diffusion and flow-matching forecasters. To ensure comparability, the baseline figures are reproduced from prior work under a shared GluonTS preprocessing pipeline, and FLOWTIME is evaluated on the same datasets, prediction lengths, and CRPS protocol.

Table 2: Test-set CRPS (lower is better) on five real-world forecasting benchmarks, comparing FLOWTIME against statistical, deep-learning, diffusion, and flow-matching baselines. CRPS is a marginal proper scoring rule, assessing per-series predictive calibration and accuracy rather than joint cross-series dependence. For FLOWTIME we report the mean $\pm$ standard error over five independent re-training and evaluation runs; baseline figures are taken from Kollovich et al. (2024) under a common preprocessing protocol. The best score per dataset is in **bold** and the second best is underlined. FLOWTIME attains the best CRPS on Electricity, Exchange, and Solar, ranks a close second on Traffic, and remains competitive on Wikipedia.

Method	Electr.	Exchange	Solar	Traffic	Wikipedia
SN	0.060 $\pm$ 0.000	0.013 $\pm$ 0.000	0.512 $\pm$ 0.000	0.221 $\pm$ 0.000	0.423 $\pm$ 0.000
ARIMA	0.344 $\pm$ 0.000	0.013 $\pm$ 0.000	0.558 $\pm$ 0.003	0.486 $\pm$ 0.000	0.654 $\pm$ 0.000
ETS	0.056 $\pm$ 0.000	0.008 $\pm$ 0.000	0.550 $\pm$ 0.000	0.492 $\pm$ 0.000	0.651 $\pm$ 0.000
DeepAR	0.051 $\pm$ 0.000	0.013 $\pm$ 0.004	0.429 $\pm$ 0.015	0.103 $\pm$ 0.000	0.215 $\pm$ 0.003
WaveNet	0.058 $\pm$ 0.008	0.012 $\pm$ 0.001	0.360 $\pm$ 0.000	0.099 $\pm$ 0.000	0.207 $\pm$ 0.000
CSDI	0.051 $\pm$ 0.000	0.013 $\pm$ 0.000	0.360 $\pm$ 0.000	0.152 $\pm$ 0.000	0.318 $\pm$ 0.012
SSSD	0.048 $\pm$ 0.000	0.010 $\pm$ 0.000	0.354 $\pm$ 0.024	0.107 $\pm$ 0.002	0.209 $\pm$ 0.000
TSFlow	0.045 $\pm$ 0.001	0.009 $\pm$ 0.001	0.343 $\pm$ 0.002	0.083 $\pm$ 0.000	0.227 $\pm$ 0.000
FLOWTIME	0.042 $\pm$ 0.001	0.009 $\pm$ 0.001	0.284 $\pm$ 0.002	<u>0.089</u> $\pm$ 0.000	0.243 $\pm$ 0.002

Results Table 2 reports CRPS for FLOWTIME against the full baseline suite on the five real-world benchmarks. FLOWTIME attains the best CRPS on three of the five datasets—Electricity, Exchange, and Solar. On Electricity it reaches a CRPS of 0.042, a 6.7% improvement over the second-best model, TSFlow (0.045). On Exchange it ties TSFlow for the best CRPS at 0.009, matching the strongest prior method while comfortably outperforming the statistical and remaining deep-learning baselines. The margin is largest on Solar, where FLOWTIME attains a CRPS of 0.284—a 17.2% reduction relative to the next-best model, TSFlow (0.343), and the single most decisive gain in the table. On Traffic, FLOWTIME ranks a close second at 0.089, marginally behind TSFlow (0.083) yet ahead of every other baseline; on Wikipedia it remains competitive at 0.243, trailing only WaveNet (0.207) and SSSD (0.209). The gains are therefore consistent but not uniform: FLOWTIME is the best or second-best method on four of the five benchmarks, and its improvements over the strongest flow-matching baseline, TSFlow, are pronounced on some datasets (Solar, Electricity) while narrow or absent on others (Traffic, Wikipedia). Read against the controlled dynamical-systems ablation, this pattern indicates that the advantages of autoregressive flow matching transfer to challenging real-world multivariate forecasting, delivering state-of-the-art marginal calibration on several benchmarks while remaining within a narrow margin of the best method on the remainder.

5 DISCUSSION

We introduced FLOWTIME, a probabilistic forecasting model that couples autoregressive factorization with flow matching to produce accurate and well-calibrated forecasts for multivariate timeseries. By decomposing the forecasting problem into a sequence of per-step conditional distributions, each realized by a shared flow-matching transport, FLOWTIME reconciles the sequential structure of temporal data with the distributional expressiveness of transport-based generative models: it can represent complex, multi-modal per-step predictive distributions while retaining the extrapolation, compactness, and calibration characteristic of autoregressive models. Our experiments substantiate this design along two complementary axes. The controlled ablation on stochastic dynamical systems isolates the autoregressive factorization as the decisive factor, since the two models differ in nothing else: the factorization improves point accuracy over the non-autoregressive flow-matching baseline on every system and, most tellingly, sustains that advantage into the extrapolation regime, where the window-to-window alternative degrades sharply. On real-world benchmarks, FLOWTIME matches or surpasses strong statistical, deep-learning, diffusion, and flow-matching baselines, attaining state-of-the-art CRPS on three of five datasets and top-two performance on a fourth. Together, and with the synthetic evidence for out-of-distribution horizons in view, these findings indicate that the autoregressive decomposition confers robust generalization beyond the training horizon—a property of direct consequence for deployment, where test conditions routinely depart from those seen during training.

These strengths are bounded by limitations that we state plainly and that mark natural directions for future work. First, the reported real-world evaluation uses CRPS, a marginal proper scoring rule that assesses per-series calibration and accuracy rather than joint cross-series dependence; our synthetic experiments exercise multi-modal predictive distributions directly, but a full assessment of the joint dependence structure would call for multivariate scoring rules such as the energy or variogram score, which we leave to future work. Second, the training–sampling asymmetry inherent in teacher forcing—ground-truth histories during training versus model-generated histories at inference—exposes FLOWTIME, like all autoregressive models, to potential error accumulation over long rollouts; our dynamical-systems results show that the factorization improves, rather than worsens, extrapolation NRMSE beyond the training horizon, but the real-world benchmarks use standard fixed prediction lengths and thus do not stress substantially longer rollouts. Third, the sequential character of autoregressive sampling incurs an inference-time overhead relative to fully parallelizable non-autoregressive models, which may constrain applicability in latency-critical, real-time settings. Fourth, the current implementation performs flow matching directly in the data space, a choice that may prove suboptimal for very high-dimensional or partially observed data. Each of these limitations is addressable within the same framework: multivariate scoring rules and longer-horizon stress tests, latent-space formulations of the per-step transport, and extensions to irregularly sampled observations are all natural avenues, as are applications to scientific modeling and decision-making under uncertainty. Overall, FLOWTIME advances probabilistic timeseries forecasting by demonstrating that coupling an autoregressive factorization with flow matching yields a model that is simultaneously accurate, well-calibrated, and robust in extrapolation, and, more broadly, that flow-based generative models constitute a compelling foundation for forecasting under uncertainty.

REFERENCES

- Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023.
- Juan Miguel Lopez Alcaraz and Nils Strodthoff. Diffusion-based time series imputation and forecasting with structured state space models. *arXiv preprint arXiv:2208.09399*, 2022.
- Alexander Alexandrov, Konstantinos Benidis, Michael Bohlke-Schneider, Valentin Flunkert, Jan Gasthaus, Tim Januschowski, Danielle C Maddix, Syama Rangapuram, David Salinas, Jasper Schulz, et al. GluonTS: Probabilistic and neural time series modeling in python. *Journal of Machine Learning Research*, 21(116):1–6, 2020.
- Ricky TQ Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. *Advances in Neural Information Processing Systems*, 31, 2018.
- Fatoumata Dama and Christine Sinoquet. Time series analysis and modeling to forecast: a survey. In *ArXiv:2104.00164*, 2021.
- Abhyuday Desai, Cynthia Freeman, Zuhui Wang, and Ian Beaver. TimeVAE: A variational auto-encoder for multivariate time series generation. *arXiv preprint arXiv:2111.08095*, 2021.
- Dheeru Dua, Casey Graff, et al. UCI machine learning repository. URL <http://archive.ics.uci.edu/ml>, 7(1):62, 2017.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024.
- Richard FitzHugh. Impulses and physiological states in theoretical models of nerve membrane. *Biophysical Journal*, 1(6):445–466, 1961. doi: 10.1016/S0006-3495(61)86902-6.
- Jan Gasthaus, Konstantinos Benidis, Yuyang Wang, Syama Sundar Rangapuram, David Salinas, Valentin Flunkert, and Tim Januschowski. Probabilistic forecasting with spline quantile function RNNs. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 1901–1910. PMLR, 2019.

Will Grathwohl, Ricky TQ Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. FFJORD: Free-Form continuous dynamics for scalable reversible generative models. In *International Conference on Learning Representations*, 2018.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33:6840–6851, 2020.

Yang Hu, Xiao Wang, Lirong Wu, Huatian Zhang, Stan Z. Li, Sheng Wang, and Tianlong Chen. Fm-ts: Flow matching for time series generation. In *ArXiv:2411.07506v1 [cs.LG]*, 2024. URL <http://arxiv.org/abs/2411.07506>.

Rob Hyndman, Anne B Koehler, J Keith Ord, and Ralph D Snyder. *Forecasting with Exponential Smoothing: The State Space Approach*. Springer Science & Business Media, 2008.

Rob J Hyndman and Anne B Koehler. Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22(4):679–688, 2006.

Maximilian Karl, Maximilian Soelch, Justin Bayer, and Patrick Van der Smagt. Deep variational Bayes filters: Unsupervised learning of state space models from raw data. *arXiv preprint arXiv:1605.06432*, 2016.

Marcel Kollovich, Abdul Fatir Ansari, Michael Bohlke-Schneider, Jasper Zschiegner, Hao Wang, and Yuyang Bernie Wang. Predict, refine, synthesize: Self-guiding diffusion models for probabilistic time series forecasting. *Advances in Neural Information Processing Systems*, 36:28341–28364, 2023.

Marcel Kollovich, Marten Lienen, David Lüdke, Leo Schwinn, and Stephan Günnemann. Flow matching with gaussian process priors for probabilistic time series forecasting. In *The Thirteenth International Conference on Learning Representations*, 2024.

Zhifeng Kong, Wei Ping, Jiaji Huang, Kexin Zhao, and Bryan Catanzaro. DiffWave: A versatile diffusion model for audio synthesis. *arXiv preprint arXiv:2009.09761*, 2020.

Guokun Lai, Wei-Cheng Chang, Yiming Yang, and Hanxiao Liu. Modeling long- and short-term temporal patterns with deep neural networks. In *The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval*, pp. 95–104, 2018.

R. Lefever and I. Prigogine. Symmetry breaking instabilities in dissipative systems. II. *The Journal of Chemical Physics*, 48(4):1695–1700, 1968. doi: 10.1063/1.1668896.

Bryan Lim and Stefan Zohren. Time-series forecasting with deep learning: A survey. *Philosophical Transactions of the Royal Society A*, 379(2194):20200209, 2021.

Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2022.

Xingchao Liu, Chengyue Gong, et al. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2022.

Edward Lorenz. Deterministic nonperiodic flow. *Journal of Atmospheric Sciences*, 20(2), 1963.

Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, 1925.

Caspar Meijer and Lydia Y Chen. The rise of diffusion models in time-series forecasting. *arXiv preprint arXiv:2401.03006*, 2024.

Adam Polyak, Amit Zohar, Andrew Brown, Andros Tjandra, Animesh Sinha, Ann Lee, Apoorv Vyas, Bowen Shi, Chih-Yao Ma, Ching-Yao Chuang, et al. MovieGen: A cast of media foundation models. *arXiv e-prints*, pp. arXiv–2410, 2024.

Kashif Rasul, Abdul-Saboor Sheikh, Ingmar Schuster, Urs Bergmann, and Roland Vollgraf. Multivariate probabilistic time series forecasting via conditioned normalizing flows. *arXiv preprint arXiv:2002.06103*, 2020.

-
- Kashif Rasul, Calvin Seward, Ingmar Schuster, and Roland Vollgraf. Autoregressive denoising diffusion models for multivariate probabilistic time series forecasting. In *International Conference on Machine Learning*, pp. 8857–8868. PMLR, 2021.
- David Salinas, Michael Bohlke-Schneider, Laurent Callot, Roberto Medico, and Jan Gasthaus. High-dimensional multivariate forecasting with low-rank gaussian copula processes. *Advances in Neural Information Processing Systems*, 32, 2019.
- David Salinas, Valentin Flunkert, Jan Gasthaus, and Tim Januschowski. DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36(3): 1181–1191, 2020.
- Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International Conference on Machine Learning*, pp. 2256–2265. PMLR, 2015.
- Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.
- Ella Tamir, Najwa Laabid, Markus Heinonen, Vikas Garg, and Arno Solin. Conditional flow matching for time series modelling. In *ICML Workshop: Structured Probabilistic Inference and Generative Modeling*, 2024.
- Yusuke Tashiro, Jiaming Song, Yang Song, and Stefano Ermon. CSDI: Conditional score-based diffusion models for probabilistic time series imputation. *Advances in Neural Information Processing Systems*, 34:24804–24816, 2021.
- Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Hugué, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport. *arXiv preprint arXiv:2302.00482*, 2023.
- Aaron Van Den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, Koray Kavukcuoglu, et al. WaveNet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*, 12, 2016.
- B. van der Pol. On ‘relaxation-oscillations’. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 2(11):978–992, 1926. doi: 10.1080/14786442608564127.
- Ronald J Williams and David Zipser. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1(2):270–280, 1989.
- Robert L Winkler, Javier Munoz, José L Cervera, José M Bernardo, Gail Blattenberger, Joseph B Kadane, Dennis V Lindley, Allan H Murphy, Robert M Oliver, and David Ríos-Insua. Scoring rules and the evaluation of probabilities. *Test*, 5:1–60, 1996.
- Jiexia Ye, Weiqi Zhang, Ke Yi, Yongzi Yu, Ziyue Li, Jia Li, and Fugee Tsung. A survey of time series foundation models: Generalizing time series representation with large language model. In *ArXiv:2104.00164*, 2024.

A DATASETS

A.1 DYNAMICAL SYSTEMS

We generated the training and test data for the five classical stochastic dynamical systems—Lorenz, FitzHugh–Nagumo, Lotka–Volterra, Brusselator, and Van der Pol—by numerically integrating their governing stochastic differential equations (SDEs). Each system takes the general Itô form

$$d\mathbf{x}(t) = \mathbf{f}(\mathbf{x}(t), t; \Theta) dt + \sigma d\mathbf{W}(t),$$

where $\mathbf{f}$ denotes the deterministic drift, Θ the system parameters, σ a constant diffusion coefficient, and $\mathbf{W}(t)$ a standard Brownian motion. For each system we drew 2400 random initial conditions from a uniform distribution over the range specified in Table 3 and integrated the SDE from each initial condition using the Euler–Heun scheme with a fixed step size, recording 200 equally spaced time points per trajectory. Each 200-point trajectory was partitioned into three consecutive segments—an observation window (the first 75 points), a prediction window (the next 75), and an extrapolation window (the final 50)—the last of which is withheld from training so that performance on it measures generalization beyond the training horizon. Of the 2400 trajectories, 2000 were allocated to training and 400 to testing.

Table 3: Specifications of the five stochastic dynamical systems used in our experiments. $\mathbf{f}(\mathbf{x})$ is the drift function, Θ the system parameters, σ the constant diffusion coefficient, and $p(\mathbf{x}(0))$ the distribution over initial conditions. Each system was integrated over the interval $[t_0, t_1]$ with 200 steps using the Euler–Heun method. Together the five systems span oscillatory, chaotic, and limit-cycle dynamics.

System	$\mathbf{f}(\mathbf{x})$	Θ	Σ	$p(\mathbf{x}(0))$	$[t_0, t_1]$
Lorenz	$f_1 = \sigma(x_2 - x_1)$	$\sigma = 10$	(1.5, 1.5, 1.5)	$\mathcal{U}([0, 10])$	[0, 2]
	$f_2 = x_1(\rho - x_3) - x_2$	$\rho = 28$			
	$f_3 = x_1 x_2 - \beta x_3$	$\beta = \frac{8}{3}$			
FitzHugh–Nagumo	$f_1 = x_1 - \frac{x_1^3}{3} - x_2 + I$	$a = 0.7$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 10]
	$f_2 = \frac{x_1 + a - b x_2}{\tau}$	$b = 0.8$			
		$\tau = 12.5$			
Lotka–Volterra		$\alpha = 1.3$	(1.5, 1.5)	$\mathcal{U}([0, 5])$	[0, 20]
	$f_1 = \alpha x_1 - \beta x_1 x_2$	$\beta = 0.9$			
	$f_2 = -\delta x_2 + \gamma x_1 x_2$	$\gamma = 0.8$			
Brusselator		$\delta = 1.8$	(1.5, 1.5)	$\mathcal{U}([0, 2])$	[0, 20]
	$f_1 = A + x_1^2 x_2 - (B + 1) x_1$	$A = 1.0$			
	$f_2 = B x_1 - x_1^2 x_2$	$B = 3.0$			
Van der Pol	$f_1 = x_2$	$\mu = 0.1$	(1.5, 1.5)	$\mathcal{U}([-2, 2])$	[0, 20]
	$f_2 = \mu(1 - x_1^2) x_2 - x_1$				

Table 4: Properties of the real-world datasets used in our experiments: series dimension n , domain Ω , sampling frequency, total number of training timesteps, and prediction length. The datasets span two orders of magnitude in dimensionality ($n = 8$ to $n = 2000$) as well as both hourly and daily frequencies, and thereby exercise the model across a broad range of multivariate forecasting conditions.

Dataset	n	Ω	freq.	timesteps	prediction length
Electricity	370	$\mathbb{R}^+$	hourly	5833	24
Exchange	8	$\mathbb{R}^+$	daily	6071	30
Solar	137	$\mathbb{R}^+$	hourly	7009	24
Traffic	963	(0, 1)	hourly	4001	24
Wikipedia	2000	$\mathbb{N}$	daily	792	30

A.2 REAL-WORLD DATASETS

To evaluate model performance we used five datasets from GluonTS (Alexandrov et al., 2020), adopting the preprocessed versions of Salinas et al. (2019) so as to ensure comparability with the baseline figures reported in prior work. Their properties are summarized in Table 4.

B NON-AUTOREGRESSIVE FLOW MATCHING BASELINE

Problem Setting We retain the forecasting setup of Section 3: given past observations $\mathbf{Y}_l$ and covariates $\mathbf{C}$, the objective is to model the conditional distribution $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$. In contrast to the autoregressive factorization of FLOWTIME, this baseline models the joint distribution of the entire future trajectory $\mathbf{Y}_f \in \mathbb{R}^{f \times n}$ directly, without any temporal factorization,

$$p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C}) = p(\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}|\mathbf{Y}_l, \mathbf{C}) \quad (9)$$

This formulation preserves the full cross-temporal correlation structure of the future window, but at the cost of having to learn a single high-dimensional distribution over $\mathbb{R}^{f \times n}$ —precisely the burden that the autoregressive factorization is designed to avoid.

Training We construct a probability path $(p^s(\mathbf{Y}_f))^{0 \leq s \leq 1}$ transporting samples from a prior $p^0(\mathbf{Y}_f) = \mathcal{N}(0, \Sigma)$ to the target $p^1(\mathbf{Y}_f) \approx p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$, where Σ is a block-diagonal covariance matrix. This prior admits the interpretation of independent per-dimension Brownian-motion processes, each a stochastic process $W: [\tau + 1, \tau + f] \rightarrow \mathbb{R}^n$; the block-diagonal structure of Σ thus imposes temporal correlation within each dimension while leaving the dimensions mutually independent.

We specify the conditional probability path as a linear-interpolation bridge perturbed by Brownian noise,

$$p^s(\mathbf{Y}_f|\mathbf{z}) = \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma) \quad (10)$$

where $\mathbf{z} = (\mathbf{Y}_f^0, \mathbf{Y}_f^1)$ with $\mathbf{Y}_f^0 \sim p^0$ and $\mathbf{Y}_f^1 \sim p^1$. Differentiating the bridge with respect to s yields the corresponding conditional velocity field,

$$\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0 + \frac{\sigma^2(1-2s)}{2}\Sigma^{-1}(\mathbf{Y}_f - ((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1)) \quad (11)$$

which in the limit $\sigma^2 \rightarrow 0$ reduces to the straight-path velocity $\mu(\mathbf{Y}_f, s|\mathbf{z}) = \mathbf{Y}_f^1 - \mathbf{Y}_f^0$, recovering the same optimal-transport interpolant used in the autoregressive model, but now defined over the full trajectory rather than a single step. We learn a neural velocity field ν_θ acting on the entire future trajectory, encoding the context in analogy with Eq. (7) as

$$\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f}) \quad (12)$$

where ζ_ϕ now ingests both past observations and covariates over the joint window. The training objective is then

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim \pi_{0,1}, s \sim \mathcal{U}(0,1), \mathbf{Y}_f^s \sim p^s(\mathbf{Y}_f|\mathbf{z})} \left\| \mu(\mathbf{Y}_f^s, s|\mathbf{z}) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s) \right\|^2 \quad (13)$$

where ν_θ is realized as a sequence model (for example a Transformer or an RNN) operating over the entire trajectory. Sampling then amounts to integrating the trajectory-level ODE,

$$\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s), \quad \psi(\mathbf{Y}_f, 0) \sim p^0 \quad (14)$$

with a numerical solver, which produces joint samples from $p(\mathbf{Y}_f|\mathbf{Y}_l, \mathbf{C})$ in a single pass over the whole horizon, without any autoregressive decomposition.

Implementation Details We employed this baseline in the dynamical-systems experiments. Here ν_θ is a four-layer bidirectional LSTM with 128 hidden units, and all remaining training hyperparameters match those of Section 4.1, so that the comparison reported in Table 1 varies the factorization alone. Training to convergence took approximately 2 minutes on a single NVIDIA A100-SXM4-40GB GPU.

Algorithm 3 Non-Autoregressive Flow Matching: Training

Require: Dataset $\mathcal{D}$, networks ν_θ, ζ_ϕ
1: **while** not converged **do**
2: Sample $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$ and $(\mathbf{Y}_f^1, \mathbf{Y}_l, \mathbf{C}) \sim \mathcal{D}$
3: Set $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C}_{\tau-l:\tau+f})$
4: Sample $s \sim \mathcal{U}(0, 1)$
5: $\mathbf{Y}_f^s \sim \mathcal{N}((1-s)\mathbf{Y}_f^0 + s\mathbf{Y}_f^1, \sigma^2 s(1-s)\Sigma)$
6: Compute target $\mu(\mathbf{Y}_f^s, s|\mathbf{z})$
7: Update θ, ϕ to minimize $\|\mu(\cdot) - \nu_\theta(\mathbf{Y}_f^s, \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)\|_{\Sigma^{-1}}^2$
8: **end while**

Algorithm 4 Non-Autoregressive Flow Matching: Sampling

Require: $\mathbf{Y}_l, \mathbf{C}$, networks ν_θ, ζ_ϕ
1: Encode context: $\mathbf{h} = \zeta_\phi(\mathbf{Y}_l, \mathbf{C})$
2: Sample initial trajectory $\mathbf{Y}_f^0 \sim \mathcal{N}(0, \Sigma)$
3: Solve $\frac{d\psi(\mathbf{Y}_f, s)}{ds} = \nu_\theta(\psi(\mathbf{Y}_f, s), \mathbf{h}, \mathbf{C}_{\tau+1:\tau+f}, s)$
4: Return $\psi(\mathbf{Y}_f, 1)$

Key Differences from the Autoregressive Variant The non-autoregressive baseline differs from the autoregressive formulation of FLOWTIME along three principal axes, which together define the trade-off that our experiments probe. The first concerns the treatment of temporal structure: the autoregressive model factorizes the joint distribution through a Markovian decomposition across time steps, whereas the baseline treats the full future trajectory $\mathbf{Y}_f$ as a single high-dimensional random variable, preserving all cross-temporal dependencies at the price of estimating a substantially more complex distribution over $\mathbb{R}^{n \times f}$. The second concerns computational profile: the baseline propagates entire trajectories through a sequence model to evaluate the velocity field, which enables parallel generation of all future time points but incurs higher memory cost, whereas the autoregressive model operates over a bounded window at each step. The third concerns the treatment of uncertainty: the autoregressive model renders the forward propagation of predictive error explicit through its chain of conditional distributions, whereas the baseline captures joint uncertainty through trajectory-level sampling but offers no explicit account of error-accumulation dynamics. These contrasts constitute complementary trade-offs—the autoregressive variant furnishes interpretable uncertainty quantification and efficient window-based computation under a bounded Markovian assumption, while the baseline retains full temporal correlations at greater computational cost and with less explicit uncertainty dynamics. Our results indicate that, for the systems and horizons studied, the autoregressive factorization is the more favorable operating point, and markedly so under extrapolation.

C EVALUATION METRICS

This section defines the two metrics used to evaluate forecast quality, chosen so as to assess probabilistic calibration and point accuracy as separate axes. Given an n -dimensional time series $\mathbf{Y}_f = \{\mathbf{y}_{\tau+1}, \dots, \mathbf{y}_{\tau+f}\}$ together with the predictive distribution—or samples drawn from it—produced by the model, we report the Continuous Ranked Probability Score (CRPS) and the Normalized Root Mean Squared Error (NRMSE), which we estimate from the sampled forecast trajectories.

C.1 CONTINUOUS RANKED PROBABILITY SCORE (CRPS)

The CRPS is a proper scoring rule widely used to evaluate probabilistic forecasts, rewarding predictive distributions that are simultaneously sharp and calibrated. As a marginal scoring rule it assesses per-series predictive quality rather than joint cross-series dependence. For a univariate random variable X with cumulative distribution function (CDF) F_X and an observed value x , it is defined as

$$\text{CRPS}(F_X, x) = \int_{-\infty}^{+\infty} (F_X(y) - \mathbf{1}\{y \geq x\})^2 dy, \quad (15)$$

where $1\{\cdot\}$ denotes the indicator function. The CRPS thus measures the discrepancy between the entire predictive CDF and the observation x , so that lower values indicate predictive distributions that are at once better calibrated and more accurate. For time series forecasting, we evaluate the CRPS at each step of the horizon and average over the horizon,

$$\text{CRPS}_{\text{avg}} = \frac{1}{f} \sum_{k=1}^f \text{CRPS}(F_{X_{\tau+k}}, y_{\tau+k}), \quad (16)$$

where $F_{X_{\tau+k}}$ is the predicted CDF—or its empirical counterpart estimated from samples—at horizon k , and $y_{\tau+k}$ is the corresponding ground-truth observation.

C.2 NORMALIZED ROOT MEAN SQUARED ERROR (NRMSE)

The Root Mean Squared Error (RMSE) is a standard measure of point-forecast accuracy, computed here from the predictive mean of the sampled forecasts. For predictions $\{\hat{y}_{\tau+1}, \dots, \hat{y}_{\tau+f}\}$ and ground-truth values $\{y_{\tau+1}, \dots, y_{\tau+f}\}$, it is given by

$$\text{RMSE} = \sqrt{\frac{1}{f} \sum_{k=1}^f (\hat{y}_{\tau+k} - y_{\tau+k})^2}. \quad (17)$$

To render the RMSE scale-invariant and thus comparable across datasets and series of differing magnitude, we adopt the Normalized RMSE (NRMSE), which normalizes by the standard deviation σ_y of the ground-truth series,

$$\text{NRMSE} = \frac{\text{RMSE}}{\sigma_y}, \quad (18)$$

where σ_y is the sample standard deviation of the observations; depending on the application, one may instead normalize by the data range ($\max(y) - \min(y)$). A lower NRMSE thus reflects more accurate predictions relative to the intrinsic variability of the series.